

Unified Substrate Definition

Numerical kernel, corrections and validation

An explicit, finite numerical specialization of the unified v0.2 formalization.

The wavefront, editable operator field, Klein carrier, binary routes, four-stage evolution and inverse/history channels remain one connected recurrence.

What this edition adds

Portable FP32 arithmetic, exact mutable operator textures, bounded execution on desktop and mobile GPUs, and measured limits of the implemented formalization.

Tom Klootwijk

Dawnwood Interactive

23 September 2026

Source lineage: original discussion -> Unified v0.2 -> numerical kernel v0.3 -> validated v0.4 -> Unified / kernel v0.5. Source PDFs are preserved unchanged.

Unified Substrate Definition, version 0.5.0

Author attribution: Tom Klootwijk, Dawnwood Interactive. **Edition:** 23 September 2026; measured numerical revision. **Numerical profile:** DWI-N1-0.5, C++ and Vulkan compute. **Validation status:** Recorded CPU/GPU comparisons pass on both physical targets at unchanged tolerances. The scope and architectural limits below are part of the result.

This edition develops the complete unified definition in *Dawnwood_Interactive_Unified_v0.2.pdf*, through the reviewed v0.4 numerical edition. It retains the connected wavefront, situated operators, non-orientable carrier, branching, history and feedback. Version 0.5 makes the mutable operator LUT an actual integer texture, shares the core descriptors within each workgroup, introduces explicitly shared elementary-function arithmetic, and separates measured capacity from nominal memory size. The v0.4 documents and PDFs remain unchanged.

References marked **Source** identify pages of the unchanged 24-page *source/double-slit-theory.pdf*. **Baseline** identifies the latest 18-page unified v0.2 formalization. Source relationships, numerical choices and established results are different categories. This document specifies the first two. The accompanying validation appendix records only work actually performed.

Measured status at a glance

Both physical devices execute the current v0.5.0 recurrence. Their listed local CPU/GPU comparisons have **zero bitwise word differences**, stronger than the unchanged float tolerance. This validates the implemented numerical bindings over those workloads; it does not validate every claim in the source proposal.

Measure	GTX 1650 Ti laptop	POCO X7 Pro / Mali-G720 MC7
Current-build bitwise comparison scope	257 states × 1,024 epochs on monolithic and staged paths; 65,537 × four on staged path	128 × 16, 257 × 1,024, 4,096 × 256 and 65,537 × four
4,096 states × 64 epochs: median GPU time	0.157168 s	0.169310 s
Mean time per whole-population epoch	2.4557445 ms	2.6454749 ms
Million state updates/s at that workload	1.667926	1.548304
Largest completed population, two epochs each	12,244,544 full states	14,569,792 full states
Two GPU state copies at that population	2,989.39 MiB	3,557.08 MiB of shared system RAM
Million state updates/s at that population	1.738059	2.445501
Validation-layer coverage	Core/synchronization enabled for comparisons; zero errors/warnings	Layer disabled; numerical comparisons still execute

Matched-workload timings are medians of three fresh processes, measured by GPU timestamps. They exclude setup, uploads, readback and host gaps. The Windows executable is a Release build; the Android APK uses its debug native build. Ordinary desktop/USB-charging conditions and sample variation are retained, so these are observations of these builds rather than peak hardware ratings. Capacity runs advance every counted record and complete full readback/health/digest checks under a changing memory policy; they are not absolute maxima or

CPU comparisons of the entire capacity population. Detailed wall times, policies, artifact hashes and raw reports are in Appendix A and `VALIDATION_v0.5.md`.

Claim boundary	Status
Fully self-defining algorithms and a self-growing tree	Partly implemented: mutable scalar bodies/parameters/positions work; native RK4, Hadamard, primitive/wrapping rules, interpreter, update order and configured tree/population remain fixed
Bitwise replay across Windows and Android from the same CLI seed	Not guaranteed: host-native initialization can differ; local comparisons start from a common snapshot
Permanent hardware-cache residence, optimal GPU use, universality or physical quantum claims	Not established by these measurements
Automatic noise deletion or arbitrary lossless one-bit distance/full-state packing	Not established; generic forms have retained numerical/storage counterexamples

1. The unified Dawnwood definition

Dawnwood is a self-referential spatial computing proposal: the acting operator field belongs to the state that returns as the next input. The double pinion, Hadamard hinge, log-polar coordinates, geometric LUT, implicit binary routes, one-bit controls and RGBA roles participate in that circulation. Optional output is downstream. [Source pp. 3-16; Baseline p. 3.]

For N wavefront records and M operator records, write the whole state as $S_n=(K,W_n,L_n,C_n)$. K is the fixed carrier in this numerical profile; W contains wavefront and kinematic variables; L contains mutable situated definitions; C identifies the channel/history/inverse roles already stored with W . It is not another independent copy of memory.

The executable recurrence is:

$$\begin{aligned} L_{(n+1)} &= J(L_n, W_n; \text{config}, n) \\ W_{(n+1)} &= E(W_n, L_{(n+1)}; \text{config}) \\ S_{(n+1)} &= (K, W_{(n+1)}, L_{(n+1)}, C_{(n+1)}) \end{aligned}$$

One logical PSI interval is mutation followed by evolution. A monolithic evolution shader or the staged ARM schedule in section 12 implements that same interval. The source's designation *Universal Spatial State Automaton* remains its proposed name; it is not a universality theorem or a physical quantum-device classification.

The executable currently realizes a **mutable descriptor field interpreted by fixed native algorithms**. The LUT changes scalar bodies, geometric parameters, positions and controls. It does not yet encode the full integrator, Hadamard matrix, primitive formulas, carrier identification, update schedule or tree-allocation rules as replaceable geometric programs. Thus the stronger source proposal that all computation be defined and rewritten within that field is only partly implemented. This distinction is architectural, separate from numerical accuracy or physical claims.

2. Source progression and notation

The source introduces double pinion and log-polar encoding on pp. 3-4; parity on pp. 4-5; the literal FT vector on p. 6; fourth-slot Y-up on p. 7; geometry, colon, phase divergence and PSI on pp. 8-9; RGBA and inverse T on pp. 9-12; the common Klein field and operator change on pp. 12-16; GPU organization and hypothetical capacities on pp. 18-23. [Baseline p. 4.]

User proposals establish the requested architecture. Earlier generated responses add stronger assertions about physics, noise removal, memory, optimization and universality. They supply neither experimental observations nor proofs.

Here n is a uint32 logical epoch; d is route depth; σ is a transported orientation bit; (u,v) is a chart representative; ρ is logarithmic radius; θ is its phase/angle variable. T_shape is a primitive, while $T_transform$ is a matrix. The golden ratio is used only to define the initialization angle. PSI is not a measured wall-clock or physical duration.

In equations below, $B_i(x)$ evaluates operator i 's scalar body, $F_i(s)$ evaluates its situated field, and $A_i(s,x)=B_i(x+0.01 F_i(s))$. These definitions refer to the current LUT unless an old LUT is stated explicitly.

3. Double pinion, mitosis and the one-bit split

The source wavefront $[0,2,0,1]$ is bound to two complex amplitudes $a=0+2i$ and $b=0+i$. Their squared norm is five; initialization does not normalize it to one. [Source pp. 3-6; Baseline p. 5.]

With g the final situated geometry response, the hinge/phase action is:

```
H = (1/sqrt(2)) [[1,1],[1,-1]]
p = phase_1 + 0.01 A_1(s',g) + epsilon*(2j-1)
  + 0.02 A_17(s',g) + 0.01 A_15(s',energy_old)
  + 0.01 A_24(s',g)
(a',b') = diag(exp(i*p),exp(-i*p)) H (a,b)
```

The displayed ideal Hadamard and complex-phase transform preserves the existing norm in exact real arithmetic. The executable uses FP32 arithmetic and approximated sine/cosine values; their squared sum need not be exactly one. Thus exact conservation is not an implementation guarantee, even if polynomial arithmetic were evaluated without roundoff. Norm error is measured separately, and there is no corrective normalization. The pinion also provides chart transport parameters du and dv , a distinct action coupled through the same records.

Population parity is $\text{popcount}(x) \bmod 2$; integer even/odd is $x \& 1$. Both give $(1,0)$ for $(7,54)$, but differ for 3. Routing consumes actual integer controls. Neither parity nor a classical complex-number calculation constitutes a physical measurement.

4. Log-polar encoding and the FT wavefront

The numerical binding is $\rho=\ln(r)$, $r=\exp(\rho)>0$. Locally, $q=(r \cos(t), r \sin(t))$, where $t=\theta+0.01 A_2(s,\rho)$. Multiplying radius by $k>0$ adds $\ln(k)$ to ρ . No quantized radius table or arbitrary information-capacity increase follows from this identity. [Source pp. 3-4, 6, 11-12; Baseline p. 6.]

The assignment $756 \rightarrow [0,2,0,1]$ is retained as a literal source instance. The source supplies no general timestamp encoder. Independently interpreting those four numbers as samples in an unnormalized forward DFT yields $[3,-i,-3,i]$; this interpretation is not substituted for the supplied initial amplitudes.

Coordinates and operator fields use FP32. No opacity clamp is applied to ρ . Exponential overflow, underflow, accumulated angle error and finite bytecode arithmetic remain possible outside a suitable operating range; finite input values alone do not prove bounded future trajectories.

5. PSI, jitter and the fourth RK4 slot

The integer generator is xorshift32: sequentially XOR x with $x \ll 13$, $x \gg 17$ and $x \ll 5$, using uint32 arithmetic. For a wavefront, $r_next = \text{RNG}(r_old \text{ XOR seed_20 XOR program_20})$, and j is its low bit when jitter is enabled, otherwise zero. This is deterministic control, not a specified physical random process. [Source pp. 7-9, 14-16; Baseline p. 7.]

Define $h = dt * (1 + 0.01 A_29(s, 0))$ and $\delta = A_13(s, \theta - 2 \theta_previous + \theta_previous_previous)$. Delta is an unscaled second difference over logical epochs, not a time-normalized second derivative. The authored PSI body modulates h ; positive base dt alone does not guarantee positive h for every edited body.

For $x = (u, v, \rho, \theta)$, with other state fields held during stages:

```
k1 = f(x)
k2 = f(x + h*k1/2)
k3 = f(x + h*k2/2)
y = yup * A_16(s, 1)
k4 = f(x + h*k3 + (0, 2*y, 0, 0))
x' = x + h*(k1 + 2*k2 + 2*k3 + k4)/6
```

The CPU and monolithic implementation use a four-iteration source loop with one shared slope helper; GLSL adds a non-unroll hint, which is not a guarantee about device optimization. The staged ARM path invokes that same slope helper in four sequential dispatches. Both retain all four slopes and perform two separate additions of y to the fourth stage. The pure Y-up expression is prepared from the original point with its updated delta, then applied only to the fourth stage. This explicitly binds the source's fourth-slot event to a position before derivative evaluation. Baseline equation 13 left the action symbolic. Ordinary RK4 convergence order and alleged noise cancellation are not established for this modified rule.

With jitter disabled, $j=0$ still gives a negative epsilon offset and negative mutation sign. This is a fixed-bit ablation. With feedback disabled, operator mutation freezes; wavefront/history recurrence continues.

6. Geometric LUT, double dot and phyllotaxis

The primitive group remains connected to the derivative: T is the minimum of two box fields; pyramid is a triangular section extruded through $|z| \leq 0.06$; circle is $\sqrt{xx+yy}$ -radius; cone uses capped-cone side/base segment distances with an inside sign; sphere is Euclidean radius minus operator radius; apex is distance to the origin. [Source pp. 8-9; Baseline p. 8.]

The apex is unsigned. A union, blend or transformed scalar field need not remain an exact metric signed distance. For each primitive, F_i applies the current scalar body to its local field. Geometry is $g = A_0(s, (F_7 + \dots + F_12)/6)$, using explicit primitive kinds 0-5.

Let $P = [[du, dv], [dv, du]]$ from pinion 5 and $Q = [[qx, qy], [qy, qx]]$ from the decoded polar vector. The colon binding is $c = A_6(s, Q:P)$, where $Q:P = 2(qxdu + qydv)$. Set $b = A_14(s, g + \text{coupling_6} * c + 0.01 A_4(s, \delta))$. For selected record t , $z = A_t(s, b)$, except downstream markers 25/26 use $z = b$. The derivative is:

```
u_dot = du_5 + 0.001 A_5(s, response)
        + 0.02 sin(theta + 2*pi*v) + fieldScale*sin(z)
v_dot = dv_5 + 0.02 cos(2*pi*v + theta)
        + fieldScale*sin(z + phase_2)
rho_dot = radialRate*(sin(theta + phase_t) - 0.1*rho)
        + 0.001*response
theta_dot = phaseRate + 0.02 A_3(s, z) + 0.01*delta
```

For downstream markers, phase_t in rho_dot is zero. The live phyllotaxis body affects b . Initialization separately uses golden angle $\gamma = 2.39996322972865332$ and square-root radial progression. This does not establish uniform Klein surface-area sampling or optimal GPU locality.

7. The Klein bottle as the common operator field

The numerical carrier is the quotient $(u+1,v) \sim (u,-v)$, $(u,v+1) \sim (u,v)$. With $m = \text{floor}(u)$, canonicalization maps u to $u-m$, v to $\text{frac}((-1)^m v)$, and σ to $\sigma \text{ XOR parity}(m)$. [Source pp. 12-16; Baseline p. 9.]

For inspection, $U=2\pi u$ and $V=2\pi v$ define the four-dimensional embedding:

$$E(u,v) = ((2+\cos(V))\cos(U), (2+\cos(V))\sin(U), \sin(V)\cos(U/2), \sin(V)\sin(U/2))$$

The displayed real-valued map obeys $E(u+1,v)=E(u,-v)$. FP32 evaluation need not make the two representations bitwise equal. This is not a globally signed solid boundary in ordinary three-dimensional space.

Each operator supplies an anchor (a,b) . A field query first canonicalizes a local copy of the wavefront. It minimizes $xx+yy$ over $m=-1,0,1$, where $x=u-a-m$ and $y=v-(-1)^m b$ rounded to its nearest vertical translate. *Strict comparisons retain the first horizontal minimizer; a vertical +0.5 tie maps to -0.5. The primitive receives $x+0.01r\cos(\theta)$, $y+0.01r\sin(\theta)$, and $z=(-1)^\sigma 0.2*(r-0.35)$.*

In v0.4, floor-based floating parity avoids undefined large float-to-uint seam conversions. A remainder rounded to exactly one is represented by the greatest FP32 value below one. Field queries canonicalize transient RK stages before the three-image search; they do not rewrite stored stage slopes.

This profile defines a canonical-chart discrete recurrence. Smooth quotient-compatible flow would additionally require $f(u+1,-v)=\text{diag}(1,-1)f(u,v)$, and consistent operator-frame transport for asymmetric primitives. The present derivative and fields do not establish those properties. Correct position wrapping must not be advertised as a proof of smooth intrinsic dynamics. The carrier is fixed; its situated scalar operators evolve.

8. One-bit tree and zero-pointer routing

Routing uses $\text{child}(i,b)=2*i+1+b$, starting at zero. It is an implicit binary decision tree; the source's term BST does not supply a search-key comparator or ordering invariant. [Source pp. 8-9, 13-15, 22; Baseline p. 10.]

At level l , the branch XORs bit l of the old RNG word, j , wavefront orientation, the current node's low flag, population parity of `seed_28 XOR program_28`, the low bit of `flags_27 XOR program_27`, and the low bit of `seed_21 XOR program_21`. There is no modulo aliasing of a missing operator address. The complete route requires $2^{(d+1)-1} \leq M$ and $d \leq 30$.

M and d are configured for a run; $2*i+1+b$ is a native address calculation. Mutation can change the route's controls and the visited records, but it does not allocate new tree nodes, alter the child rule or grow the population. The current structure is not a self-growing tree.

The catalogue order is an implementation choice retained from the baseline:

Index	Entry	Numerical role
0	klein	Geometry aggregate
1	hadamard	Hinge phase response
2	phi	Polar phase encoding
3	rk4	Angular derivative response
4	phyllotaxis	Divergence/blend response
5	pinion	Transport and coupling
6	double_dot	Colon response
7	T_shape	Two-box primitive
8	pyramid	Triangular extrusion
9	circle	Radial section

Index	Entry	Numerical role
10	cone	Capped cone
11	sphere	Spherical field
12	apex	Point-distance field
13	delta_phi	Phase difference response
14	blend	Coupled scalar response
15	wavefront	Previous-energy phase input
16	y_up	Fourth-stage displacement
17	crystal	Output phase input
18	inverse_T	Inverse response feedback
19	T_transform	Rotation/shear parameters
20	jitter	RNG integer context
21	bst	Branch integer context
22	return	Final horizontal increment
23	phase_history	History response
24	dichromatic	Paired phase input
25	bayer	Downstream threshold readout
26	bc5	Downstream packing experiment
27	split	Branch integer context
28	parity	Population-parity context
29	psi	Local step modulation
30	mutation	Feedback mutation rate

9. Dichromatic RGBA and A as inverse T

R and G designate the complex pinion streams, requiring four FP32 amplitude components together. B designates finite history. A is an integer reference to the inverse matrix stored in the same wavefront record, not scalar opacity. These conceptual channel roles are distinct from the four physical uint32 components of an operator-texture texel in section 12. [Source pp. 9-12; Baseline p. 11.]

Set $\text{angle} = \text{theta}' + \text{phase_19} + 0.01 \text{ A_19}(\text{s}', \text{g})$, $c = \cos(\text{angle})$, $s = \sin(\text{angle})$, and $q = \text{shear_19}$. Then:

```

T = [[c, c*q-s], [s, s*q+c]]
D = c*c+s*s
T_inverse = [[s*q+c, -(c*q-s)], [-s, c]] / D
response' = A_18(s', inverse00*g + inverse01*delta)
history' = w*history_old + (1-w)*A_23(s',
    k1.v+k2.v+k3.v+k4.v+j+response')
```

The stable determinant D exploits rotation-times-shear structure. It avoids cancellation of large shear terms present in the expanded determinant. Large shear can still produce an ill-conditioned inverse; this correction is not a conditioning guarantee. The response feeds subsequent radial motion and mutation. Inversion does not inherently erase independently added channel noise.

10. The complete PSI circulation

The numerical schedule specializes the baseline's written symbolic order. [Baseline p. 12; Source pp. 13-17.]

1. Read old state and old LUT; update every operator from reproducible feedback samples.

2. Establish a compute-write to compute-read dependency.
3. Read old wavefront and new LUT; determine RNG bit and route; compute h and phase difference.
4. Evaluate four derivative stages, with double Y-up only in the fourth.
5. Add $h \cdot 0.001 \cdot A_{22}(s, \text{field_old})$ to u; canonicalize the resulting position.
6. Recompute geometry; update complex amplitudes, inverse matrix, response and history.
7. Store energy, four vertical slopes, route, reference, RNG, prior phases, jitter and embedding.
8. After every state has completed, establish the next dependency, exchange old/new state buffers and LUT images, and increment the epoch.

Thus the implemented amplitude hinge follows kinematic integration. The baseline's named actions do not establish that these numerical operations commute. This order is explicit and versioned.

Bayer reads completed state using ordered-threshold dithering. The source's optional name is ambiguous with a camera color-filter mosaic; this binding does not simulate a sensor. BC5 is a separate two-channel block-format experiment. Neither downstream marker injects compressed pixels into feedback.

11. Self-reference, operator change and expression

An operator record contains position, geometric parameters, phase, gain, coupling, shear, transport, mutation rate, program, seed, kind and flags. Its scalar program has eight four-bit instructions, low nibble first: identity, add gain, subtract gain, multiply coupling, sine, cosine, absolute value, negate, square. The default 0x341 means add, sine, multiply, then identities. Unknown instructions are invalid inputs. [Baseline p. 13.]

For operator i , feedback samples are $a = (17i + n) \bmod N$ and $b = (31i + n + 1) \bmod N$. v0.4 evaluates these mathematical indices through overflow-safe reduced uint32 additions. Old record 30 changes all records, including itself; old record 5 supplies transport.

With old states A and B and advanced operator seed, define j_i from seed XOR A.rng XOR B.route. If jitter is disabled, $j_i = 0$. Define $\text{rate} = \text{mutationRate}_{30}(1 + 0.1 \cdot A_{30}(A, A.\text{field}))$. The update is:

```
u += dt*(du_5 + rate*sin(A.theta) + 0.001 F_5(A))
v += dt*(dv_5 + rate*cos(B.theta)); canonicalize
phase += rate*(2*j_i-1) + 0.001*dt*(A.history+B.response)
gain += 0.1*rate*sin(F_i(A)+A.theta)
shear += 0.1*rate*cos(B.field)
```

The gain field query uses the operator's newly moved position. When $j_i = 1$, its first nibble toggles add/subtract or sine/cosine; other opcodes remain unchanged. Changed numerical bodies affect future action without native shader recompilation.

This is finite self-modification of descriptors and scalar expressions. The `rk4` entry changes an angular-derivative response; editing it does not replace the four-stage integration algorithm. The `hadamard` entry changes a phase response; editing it does not replace the fixed Hadamard matrix. The named geometry entries supply parameters and scalar-body responses to native primitive formulas. The carrier, wrapping rule, instruction interpreter and mutation/evolution schedule are also fixed. Those boundaries must be explicit when comparing the implementation with the source's stronger geometrically self-defining architecture. Neural, database and universal-computation mappings remain design vocabulary requiring concrete encodings and evidence.

12. GPU substrate and resident organization

The physical targets remain the owner's GTX 1650 Ti laptop with 4 GB and POCO X7 Pro 12 GB system-RAM edition, superseding the source's RTX/12 GB scenario. Both devices are under validation in the current phase; actual outcomes are in Appendix A and `VALIDATION_v0.5.md`. Actual device identity, heap sizes, feature support and execution must be reported separately. Phone system RAM is not dedicated VRAM. [Source pp. 18-19; Baseline p. 14.]

The mutable operator LUT is now two actual optimal-tiling `VK_FORMAT_R32G32B32A32_UINT` images. Each 64-byte operator occupies four consecutive `RGBA32UI` texels, with row wrapping handled by the linear texel index. Every texel has four 32-bit unsigned components. The first twelve components preserve the bit patterns of the twelve FP32 parameters; shader bitcasts recover their values. The final four components remain integer words:

Texel within operator	Four components, in order	Interpretation
0	u, v, radius, height	Four FP32 bit patterns
1	phase, gain, coupling, shear	Four FP32 bit patterns
2	du, dv, mutationRate, reserved	Four FP32 bit patterns
3	program, seed, kind, flags	Four uint32 words

This preserves full parameter precision; it is not a one-bit SDF representation. Binary controls use individual bits of `flags`, currently orientation bit 0. The full flag word is preserved. `program` still contains eight four-bit instructions because the nine-opcode vocabulary cannot fit one instruction into one bit. Integer `texelFetch` reads exact texels without interpolation, normalized conversion or lossy compression. The SDF-like fields are evaluated from these descriptors; the texture does not contain a precomputed spatial grid of distances.

Mutation samples the old LUT, performs the declared operator update, and writes the new image with `imageStore`. A compute-write to compute-read dependency precedes evolution's reads of the new LUT. Descriptor bindings 2, 3 and 4 are respectively old sampled image, new storage image and new sampled image. Old/new state storage buffers remain at bindings 0 and 1. No host upload, texture copy or state copy occurs between logical epochs; initialization and requested readback are explicit transfers.

Evolution cooperatively reads the 31 core operators into 1,984 bytes of workgroup shared storage. The workgroup has 64 lanes, and all lanes reach its synchronization barrier before out-of-population lanes return. Core references then use this shared copy; larger routing catalogues fetch additional operators from the texture. Mutation shares its two common controls, records 5 and 30, in 128 bytes. This gives explicitly allocated shared storage for that workgroup's execution. It does not prove permanent hardware texture-cache residence, any particular cache hit rate or optimal occupancy.

Each output has one writer. For populations above the current 65,536-state partition limit, the runtime advances independent evolution slices while all slices read the same old population and the same new LUT. Mutation executes once per epoch; buffer/image parity and epoch change only after all slices complete. A temporary push-constant copy uses `Config.reserved0` as the global slice offset. This offset never becomes stored configuration or checkpoint state. The partition size also respects the device's maximum workgroup count. Small populations retain batching of at most eight complete epochs per submission. These bounds reduce individual submission work; they are not a universal watchdog-time guarantee.

Staged ARM evolution

The initial v0.5 Mali pipeline could not compile successfully, including after an authorized background-app cleanup. The current implementation therefore selects smaller evolution entry points for ARM vendor ID 0x13b5; the GTX defaults to the monolithic path. DAWNWOOD_SPLIT_EVOLUTION=1 allows desktop inspection of the same staged path. The staged APK now executes on the actual POCO and passes the recorded comparisons, including 257 states $\times$ 1,024 epochs, 4,096 $\times$ 256 and 65,537 $\times$ four, with zero bitwise differences. The smaller shader architecture alone would not establish those results; the actual reports do.

Every active population partition executes the following schedule after the epoch's single LUT mutation:

Dispatch	Shared numerical function	Output
1: prepare	dw_prepare	Route/RNG controls, modulated step, phase difference and fourth-stage Y-up value
2-5: slopes	dw_slope, stages 0-3	Four four-component FP32 derivatives, each consuming its predecessor where required
6: combine	dw_combine	Integrated/canonicalized position and retained old amplitudes/history in the output state
7: geometry	dw_geometry	Situated field at the combined point
8: finish	dw_finish	Amplitudes, inverse T, feedback/history, energy, route, phase history and embedding

These are eight dispatches for one evolution, not eight logical PSI intervals. Compute-write to compute-read/write dependencies separate stages. Old state and new LUT remain common to every stage; scratch is reused only after a partition completes, and state/LUT parity still changes only after the whole population completes. CPU and monolithic GPU execution call the same helpers with the same intended FP32 expression ordering. Actual bitwise agreement must be measured; sharing source alone is not its proof.

The transient EvolutionScratch structure is 96 bytes: four FP32 scalars, four uint32 controls and four four-component slopes. Binding 5 is a device-local scratch buffer containing $\min(N, \text{stateDispatchLimit})$ such records, at most **6 MiB** for the current 65,536-state partition bound. Config.reserved1 carries the slope index only in a temporary push-constant copy. The stored Config, 128-byte state, 64-byte operator and checkpoint ABI remain unchanged. The monolithic path has no such device scratch allocation. Every staged dispatch reloads its own workgroup-shared LUT window; no workgroup memory persists across dispatches. Extra dispatches, barriers and scratch traffic require their own performance measurements.

Uploads and readbacks use a reusable staging buffer no larger than 16 MiB; image copies use complete row chunks. State descriptors still span the whole population, so maxStorageBufferRange remains a real population limit. State-buffer allocations, image requirements, padding, selected heaps and staging are reported separately. Discrete-GPU state allocations prefer non-host-visible device-local memory over a small host-visible aperture when available. Runtime allocation uses the configured startup budget fraction; the capacity runner obtains fresh probes between candidates and applies additional host-memory reserves. A heap's total size alone is not available memory.

The shader build retains real-loop DontUnroll hints, precise return temporaries and one shared RK derivative call site. These limit avoidable compiler expansion but do not guarantee a specific driver memory footprint. CPU compilation, SPIR-V validation, device pipeline creation, GPU execution and successful numerical comparison are separate evidence categories.

13. Packing scenarios and operation-level cost

A wavefront is 128 bytes: 28 FP32 values and four uint32 values. An operator is 64 bytes: twelve FP32 values and four uint32 values. Config is 64 bytes. Two state buffers require $256 \times N$ payload bytes; the two operator images contain $128 \times M$ meaningful payload bytes before image-row padding and device allocation alignment. The default 31-operator image contains 124 texels and 1,984 meaningful bytes; both image copies contain 3,968 bytes. Actual image allocation can be larger. [Source pp. 20-22; Baseline p. 15.]

The staged ARM path additionally uses $96 \times \min(N, \text{stateDispatchLimit})$ bytes of device scratch before allocation padding, up to 6 MiB. This is temporary workspace, not additional logical states or checkpoint payload. Capacity accounting includes it on the same physical-memory system as the phone's other allocations.

The old and new state copies are needed simultaneously because one epoch reads old values while writing new values. They are two copies of N evolving states, not $2N$ independent states. Bounded staging adds at most 16 MiB in the present runtime. A full host snapshot still consumes $128 \times N + 64 \times M$ bytes during initialization or readback. The ordinary GPU run releases its initial host snapshot before allocating the final one. On shared-memory hardware those CPU and GPU allocations compete for the same physical RAM; driver and pipeline allocations add further overhead.

BC5 uses 16 bytes per two-channel 4x4 block, averaging one byte per texel, with generally approximate reconstruction. It remains an optional downstream experiment and is not the exact RGBA32UI operator LUT. Neither representation silently replaces the full 128-byte recurrent state. Approximately 12.88 billion one-byte elements describe a hypothetical 12 GiB pool, not twelve billion complete wavefront records. The source's 24-48 billion operational-state scenario does not specify a complete-state byte width.

For fixed instruction width and primitive count, an epoch performs $\Theta(M+N \times d)$ work. With default fixed d , work grows linearly with N . A route costs d child operations. One lookup or shared-control change is a different workload from advancing every state. Removing two actually present uint32 pointers would save eight bytes per record; this layout already has no child pointers.

The measurements have the following plain meanings:

Metric	What it measures	What it does not establish
Completed population N	Full 128-byte records that all completed the requested epochs	Twice as many states because two GPU copies exist
State updates per second	$N * \text{completed_epochs} / \text{kernel_seconds}$; one update is one whole recurrence for one state	FLOPS, instructions/s, or independent logical operations/s
Kernel seconds	Sum of recorded GPU submission timestamps, or identified host-fence fallback	Initialization, pipeline creation and full readback cost
Whole-process seconds	Launch, initialization, pipeline work, execution and readback together	A continuously running engine's steady-state frame time
Actual allocated bytes	Padded buffers and images in their reported heaps	Guaranteed permanent physical residency or all driver allocations
Capacity result	Highest population actually completed under the recorded memory policy	An absolute hardware maximum or numerical equivalence
Memory bandwidth / occupancy / cache hit rate	Separate transfer or hardware-counter observations, if collected	Values inferable from allocated bytes alone

`tools/measure_capacity.py` measures complete kernel runs, not dummy allocation fillers. Its default policy targets 95% of freshly reported application GPU headroom, subtracts a 128 MiB reserve, and limits host pressure to 85% of currently available physical RAM minus that reserve and staging. It accounts for a transient host snapshot at 128 bytes/state on discrete hardware and both GPU copies plus that snapshot at 384 bytes/state on shared-memory hardware. Those are allocation-policy estimates, not measurements of total process memory.

The runner starts with a modest real population, grows toward device and memory limits, and refines recoverable allocation failures. Every counted candidate must finish all requested epochs, return a full-record digest and pass health checks. Serious device errors, timeouts or unhealthy states stop growth. The output records the best completed population, failed allocation bracket if any, policy, per-heap placement, raw command evidence and stopping reason. Using nearly all the permitted memory is different from proving nearly all arithmetic units or memory bandwidth are busy. The initial candidate's measured result appears in Appendix A; it must not be presented as a benchmark of later arithmetic without rerunning it.

14. Editable bindings and defaults

The following values define the default profile, rather than equations attributed to the source. [Baseline p. 16.]

Configuration	Default
N, M, epoch, depth	256, 31, 0, 4
dt, yup, epsilon	0.01, 0.0005, 0.125
mutation, jitter, feedback	1, 1, 1
phaseRate, radialRate	0.125, 0.01
historyWeight, fieldScale	0.75, 0.005
Stored/checkpoint reserved configuration words	Zero; dispatch offset uses a transient copy only

For state i , let $r_i = \sqrt{(i+0.5)/N}$ and $t_i = i\gamma$. Initialize $(u,v) = (0.5+0.45r_i \cos(t_i), 0.5+0.45r_i \sin(t_i))$, $\rho = \ln(0.2+0.4r_i)$, and $\theta = t_i \bmod 2\pi$. Prior phase samples equal θ ; inverse is identity; amplitudes and energy are the source fixture. RNG is $\text{RNG}(756 \text{ XOR } (i747796405+2891336453))$, replacing a zero result with 756. Other state values start at zero except $\alpha_{\text{ref}}=i$ and the calculated embedding.

Operator i uses $r_i = \sqrt{(i+0.5)/M}$, position $0.5+0.4r_i(\cos(t_i), \sin(t_i))$, radius $= 0.18+0.005(i \bmod 5)$, height $= 0.25$, phase $= 0.02i$, gain $= 0.1$, coupling $= 0.9$, shear $= 0.1 \sin(t_i)$, $(du, dv) = 0.03(\cos(t_i), \sin(t_i))$, mutationRate $= 0.0002$, program $= 0x341$, kind $= i$ and zero flags/reserved. Seed is $\text{RNG}(756+17*i)$, replacing zero with one.

These are mathematical initialization formulas; the current `initialize()` implementation uses host-native C++ square root, sine, cosine, logarithm, remainder and ordinary division for them. The portable arithmetic binding applies to the mutation/evolution recurrence and does not yet replace all host initialization. Identical CLI configuration and seeds therefore need not produce identical initial FP32 bits on Windows and Android. Each local CPU/GPU verification clones one initial snapshot into both backends; it validates that pair. Cross-device bitwise replay requires an identical initial-state/checkpoint payload and a separate measured comparison. Different Windows/Android run digests from locally initialized runs are not, by themselves, evidence of a GPU recurrence defect or of cross-device equivalence.

Shared equations live in `include/numeric_types.inc`, `include/numeric_evolve.inc` and `include/numeric_math.inc`; the GPU texture representation is in `shaders/operator_texture.inc`. Changing them requires regenerated shader artifacts. CLI program edits and checkpoint record edits change supported live definitions. Authoring remains

bounded by opcode, representation, finite-value and device constraints; v0.2's unbounded symbolic editing is not promised here.

15. Reproduction, checkpoints and validation boundaries

The CPU and Vulkan implementations share numerical equations. Their agreement from the same initial snapshot checks implementation, layout and scheduling; it does not independently derive those equations or verify physical claims. Host-native initialization remains a separate cross-platform reproducibility boundary, as stated in section 14. Meaningful evidence distinguishes analytic fixtures, backend comparison, checkpoint continuation, operator sensitivity, real-device measurement and external experiment. [Baseline p. 17.]

Representative commands, using the executable path appropriate to the build, are:

```
dawnwood selftest
dawnwood verify --device "1650 Ti" --count 256 --steps 16
dawnwood run --backend cpu --count 256 --steps 16
dawnwood run --resume state.dwk --steps 16 --checkpoint continued.dwk
```

These are reproduction instructions, not claims that these runs succeeded. Per-epoch comparisons should retain the first divergent field and epoch. Backend tolerance must not be relaxed merely to hide a failure. Operator sensitivity must inspect numerical state, not only the edited program word. Norm, chart health and inverse residual each address a different invariant. A run health pass denotes finite and structurally valid records; it does not impose the separate norm-error fixture threshold.

Checkpoint layout remains DWKN003: eight-byte magic, two record-size words, 64-byte Config, N state records and M operator records. Thus the 80-byte header and existing record ABI remain compatible. Valid v0.3/v0.4 checkpoints can supply initial state to v0.5 arithmetic and equations; this does not promise bitwise replay of an earlier trajectory. The checkpoint still stores ordinary 64-byte operator records, which are packed into exact integer-texture texels on upload and unpacked on readback. Unknown programs or invalid records must be rejected rather than interpreted as successful no-ops.

The v0.4 FP32-literal correction and strict noncontraction policy remain. CPU compilation uses `/fp:strict`; GLSL precise variables and return temporaries request `NoContraction` for basic arithmetic. Version 0.5 additionally specifies shared elementary-function arithmetic in `include/numeric_math.inc`, documented in `PORTABLE_MATH_v0.5.md`. This is an explicit change to the executable numerical profile, not a hidden relaxation of comparison tolerances.

Version 0.5 replaces native sine/cosine and exponential calls with ordered FP32 approximations shared by CPU and GPU. Small trig arguments use split $\pi/2$ reduction; larger finite FP32 values use integer significand reduction with a 256-bit $2/\pi$ constant. Reduced sine/cosine use degree-13/12 polynomials. Exponential uses split $\ln(2)$ reduction, a degree-8 polynomial and explicit exponent construction, including defined overflow/underflow handling. These approximate the original real functions without clipping finite angles, removing fields or normalizing amplitudes.

After the first candidate's longer GTX comparison still failed, square root and dynamic division were given shared integer-corrected rounding. A native operation now supplies only an estimate on normalized operands. Exact high/low uint32 products determine the quotient/remainder or square-root floor, then integer midpoint logic selects the round-to-nearest/even result. Division handles output subnormals without double rounding; signs, zero, infinity and NaN have explicit bit-level cases. All dynamic recurrence divisions, including division by six, use this binding. This preserves the real equations while changing their executable FP32 realization. `PORTABLE_MATH_v0.5.md` defines the complete algorithms and exceptional cases.

The shader build can preserve the six portable functions with SPIR-V `DontInline` before offline optimization, avoiding repeated expansion of their full-range arithmetic. Optional interpreter-function preservation also retains field/body/evolution helper boundaries. Shared code uses explicit non-unroll hints and loop-based request

evaluation to reduce repeated call sites without intentionally regrouping FP32 expressions. Drivers can still expand these functions; the recorded Mali failure after compaction demonstrates that small SPIR-V size is not a sufficient compilation guarantee. The staged schedule further limits each entry point's work. These are compilation-resource changes, not speedup guarantees; each accepted artifact needs its own identity and device observations.

Sampled approximation errors are not exhaustive proofs of worst-case accuracy. Constructing a subnormal result does not establish that every device preserves subnormals in subsequent arithmetic. Even when both backends follow the same intended formula, compiler arithmetic, exceptional cases and accumulated errors require device measurements. The comparison rule remains $\text{abs}(\text{cpu-gpu}) \leq 1\text{e-}5 + 2\text{e-}5 \cdot \max(\text{abs}(\text{cpu}), \text{abs}(\text{gpu}))$, with exact integer words. Bitwise backend identity and unlimited trajectories are not promised.

Runtime hardening includes complete checkpoint/snapshot validation, strict CLI floating-value parsing, rejection of a vacuous zero-epoch verification, and full configuration reporting at adequate precision. Invalid input must fail before entering the numerical recurrence. Epoch exhaustion is an explicit limit. Detailed actual outcomes, unavailable hardware and unresolved failures belong in the validation appendix and raw logs, not inferred from file presence.

Source and edition record

The unchanged 24-page source supplies the conceptual relationships. Unified v0.2 supplies this edition's complete fifteen-section baseline. Kernel v0.3 supplies DWI-N1's executable choices. Unified v0.4 supplied matching literal precision, mathematical feedback indexing without overflow, safer seam parity/remainders, canonical transient field queries, the structured inverse determinant and runtime/input validation. Unified v0.5 retains those corrections and makes operator texture storage, workgroup sharing, bounded transfers, partitioned evolution and the revised arithmetic policy explicit.

This is a new numerical revision with a compatible record layout. The connected state/operator feedback is implemented, while full geometric definition of the native algorithms and self-growing structure remain missing. Smooth transported-frame dynamics, general universality, physical double-slit advances, automatic error correction, permanent cache residence and hypothetical capacity/performance advantages remain unproved. These distinctions make the implemented subset and the remaining proposal reviewable.

Tom Klootwijk - Dawnwood Interactive - Unified Substrate Definition v0.5.0.

Appendix A. Validation record

Snapshot date: 23 September 2026. Current desktop and phone artifacts pass their recorded local CPU/GPU comparisons with zero bitwise differences. Completed benchmark/capacity runs measure 12,244,544 states on GTX and 14,569,792 on POCO, each for two epochs. The following record also preserves earlier failures and superseded artifacts; their timings are never relabeled as current-build results. `VALIDATION_v0.5.md` collects the evidence and quick claim scan. No new repository test sources were added.

Observation on the identified candidate	Result	Evidence under <code>results/v0.5/</code>
Existing CPU fixtures, initial and corrected candidates	30/30 pass on each	<code>cpu_texture_portable_v05.stdout</code> , <code>cpu_corrected_arithmetic_v05.stdout</code>
Corrected sqrt/div candidate, GTX 257 states, 1,024 epochs	All epochs pass; zero float difference and exact integer words	<code>verify_corrected_257_1024_v05.stdout</code>
Corrected candidate, GTX 257 states, 4,096 epochs	Zero bitwise word differences at every epoch	<code>verify_final_257_4096.stdout</code>
Corrected candidate, GTX 4,096 states, 256 epochs	Zero bitwise word differences at every epoch	<code>verify_final_4096_256.stdout</code>
Corrected candidate, GTX 65,537 states, four epochs	Zero bitwise differences across the evolution partition boundary	<code>verify_final_65537_4.stdout</code>
Corrected candidate, GTX 65 states, 511 operators, 64 epochs	Zero bitwise differences with depth 8 and a larger LUT	<code>verify_final_65_511_64.stdout</code>
GTX 128 states, 31 operators, 16 epochs	All requested per-epoch comparisons pass	<code>verify_texture_portable_128_16_v05.stdout</code>
Initial trig-only candidate, GTX 257 states, 1,024 requested epochs	First failure at epoch 305; 304 preceding epochs pass	<code>verify_texture_portable_257_1024_v05.stdout</code>
GTX 65 states, 511 operators, 16 epochs	Pass; includes a two-row operator texture	<code>verify_lut_65_511_v05.stdout</code>
GTX 65,537 states, two epochs	Pass across the 65,536-state partition boundary	<code>verify_chunk_boundary_v05.stdout</code>
Multirow LUT upload/readback	Exact checkpoint hash match, including high flag bits and signed zero	<code>lut_roundtrip_identity.json</code>
262,145-operator LUT transfer through bounded staging	Exact checkpoint match across the 16 MiB staging boundary	<code>manual_edge_checks_v05.json</code>
257-state, 17-epoch batching equivalence	Exact checkpoint match with requested batch 17 versus 1	<code>manual_edge_checks_v05.json</code>
Vulkan core/synchronization validation for listed GPU comparisons	Zero recorded errors and warnings; layer enabled	Corresponding command/stdout/stderr files
Initial candidate full-population capacity	12,490,752 states each complete two epochs with healthy full readback	<code>capacity_candidate1/summary.json</code>
Corrected existing claims suite	17 Python checks, 30 CPU fixtures, 29 active-operator sensitivity checks and CPU checkpoint replay pass	<code>claims_corrected/summary.json</code>
Initial 23 September v0.5 POCO build	CPU fixtures pass; GPU evolution-pipeline creation fails with <code>VkResult -3</code> ; later ordinary run terminates its process	<code>2026-09-23/phone_validation/</code>

Observation on the identified candidate	Result	Evidence under results/v0.5/
Corrected-arithmetic GTX count-scaling benchmark	Three samples/count, 16 epochs, GPU timing identified separately from CPU elapsed time	claims_corrected/benchmark_scaling.json
23 September corrected executable, GTX $257 \times 1,024$ and $65,537 \times$ four epochs	Zero bitwise differences; layers enabled, zero errors/warnings	2026-09-23/laptop/summary.json
23 September corrected executable, GTX capacity	12,256,960 states $\times$ two epochs, healthy full readback, 2.830 million updates/s	2026-09-23/laptop/capacity95/summary.json
Staged phone APK, 128 states $\times$ 16 epochs	Zero bitwise word differences; actual Mali execution, validation layers disabled	2026-09-23/phone_split/verify.json
Staged phone APK, $257 \times 1,024$, $4,096 \times 256$ and $65,537 \times$ four states/epochs	All pass with zero bitwise differences; phone layers disabled	2026-09-23/phone_extended/verify*_sep23.json
Staged phone APK, three $4,096$ -state $\times$ 64-epoch runs	Healthy, identical same-device digests; median GPU time 0.1693103924 s	2026-09-23/phone_benchmark/summary.json
Current desktop build, staged $257 \times 1,024$ and $65,537 \times$ four; monolithic $257 \times 1,024$	Zero bitwise differences; core/synchronization layers enabled, zero errors/warnings	2026-09-23/laptop/final_summary.json
Current desktop capacity	12,244,544 states $\times$ two epochs at 1.738 million updates/s under the recorded policy	2026-09-23/laptop/capacity_final95/summary.json
Current phone capacity	14,569,792 states $\times$ two epochs at 2.445501 million updates/s under the Android memory policy	2026-09-23/phone_capacity/summary.json

The initial failed long comparison remains important evidence: **epoch 305, state[205].ai**, CPU **0.0916269422**, Vulkan **0.0916389078**. One float exceeded the unchanged tolerance; there were zero integer mismatches, nonfinite values or invalid records. Aggregate maximum absolute error was **2.05039978e-5** and worst scaled error **1.011227649**. This trig-only candidate failed its requested 1,024-epoch comparison. After shared integer-corrected division and square root were added, the recorded 257-state, 1,024-epoch comparison passed with **maximum absolute error zero, worst scaled error zero**, exact integer words, and no invalid/nonfinite records. Core and synchronization validation were enabled and reported zero errors and warnings. That proves agreement for the recorded workload, not all checkpoints, devices or unlimited time horizons.

A separate strict-CPU diagnostic sampled 5,000,000 random input pairs. Its **4,961,170 finite divisions with nonzero denominators** and **4,980,524 finite nonnegative square-root inputs** produced **zero result-bit mismatches** against the host's native operations. portable_rounded_accuracy.json records this observation. portable_boundary_accuracy.json further records zero mismatches over all **16,777,216 normalized square-root inputs in [1,4)**, **2,496,400 division boundary cases**, and **1,580 square-root edge/special cases**. The arithmetic specification explains the integer rounding construction. These domains and samples are not an exhaustive all-device proof.

The multirow checkpoint roundtrip SHA-256 was **337cc0c88938f2dea0c3aeb33667c681f9b4fa9bf5b3ad3ffe43087fafbf787e**, identical before and after transfer. This supports exact representation for the exercised records; it does not measure recurrence accuracy or texture-cache residency.

The first shared sin/cos/exp implementation was sampled against double-precision host references at exact conversions of FP32 inputs. Each grid contained 2,000,001 points; trig also used 1,992,127 finite random FP32 bit patterns. Worst observed sine/cosine absolute errors were approximately **1.0882e-7 / 1.0628e-7**, or **3 / 6 ULP**

against rounded double references. Exponential's maximum observed normal-result relative error was approximately **9.4147e-8**, with at most **1 ULP** across its sampled inputs. `portable_math_accuracy.json` and `PORTABLE_MATH_v0.5.md` define the sampling and limitations. These are exploratory measurements, not exhaustive proofs or GPU agreement by themselves.

The initial candidate capacity run in `capacity_candidate1/` completed **12,490,752 full states**, each for **two epochs**, and returned healthy finite/chart-valid records. The two GPU state copies used **3,049.5 MiB** of payload. Kernel device timestamps summed to **1.006620256 seconds**, corresponding to **24.817 million state updates/second**; whole-process time was **22.969 seconds**. The run used 382 compute submissions, the longest **3.05962 ms**. Total reported buffer/image allocation was **3,214,441,472 bytes**, including the 16 MiB staging allocation; these totals omit driver/compiler allocations. The full-state/LUT digest was `ce1eb05b401c0a1a`.

This earlier capacity run stopped at its **95%-of-current-budget minus 128 MiB** policy boundary, not at an allocation failure or an absolute hardware maximum. Its Vulkan validation layer was disabled, so zero layer counters do not constitute a validation-layer result. It measured the trig-only candidate; the separately identified corrected-build measurement follows below. Completion and healthy records do not independently certify CPU/GPU numerical equivalence.

The **23 September corrected-arithmetic executable** now has its own capacity result: **12,256,960 states × two epochs**, **2,992.42 MiB** state ping-pong payload, **8.661623776 seconds** of GPU device time, **2.830176 million state updates/s**, and **37.266 seconds** for the whole process. Maximum recorded submission is **24.454 ms**. Full readback, digest and health checks complete; the policy remains 95% of reported free device budget minus 128 MiB, with no allocation failure. The full population was not CPU-compared. Its executable SHA-256 is **20b944a181d52e105150196b1f7e2a4f309b79cc478ebd9185f161e9686085e0**, retained with its shaders in `2026-09-23/laptop/pre_compaction_artifacts/`. This is measured before a subsequent phone compiler-compaction experiment; later accepted artifacts need separate attribution and validation. The corrected arithmetic is materially slower than the earlier trig-only capacity candidate; the old 24.817-million rate is not its performance.

For the same identified executable, a **4,096-state × 64-epoch** workload has three-sample median **0.09632352 seconds** GPU device time, **1.505055 ms per whole-population epoch**, **2.721495 million state updates/s**, and **0.688 seconds** whole-process time. CPU medians are **6.4672045 seconds** evolution wall time and **6.5 seconds** whole-process time. All final state/LUT digests agree. GPU samples range from 0.090845888 to 0.153940768 seconds; normal desktop conditions and startup/clock variation are retained rather than hidden. CPU wall time, GPU timestamps and full-process timing have different scopes. `VALIDATION_v0.5.md` and `laptop/benchmark_summary.json` record those distinctions.

The established v0.4 results remain in the unchanged `FORMALIZATION_v0.4.md`, `CHANGES_v0.4.md` and `results/v0.4/`. In particular, its GTX epoch-96 and POCO epoch-1 numerical failures are historical results for that arithmetic. ADB now identifies POCO X7 Pro, model 2412DPC0AG, with Mali-G720 MC7. Android reports 11,861,921,792 bytes of system RAM; available RAM changes with system pressure. The initial v0.5 probe/verification return `vkCreateComputePipelines` evolve ... (`VkResult -3`), and a later ordinary run terminates its native process. Subsequent compaction and authorized background-app cleanup still fail to compile the monolithic pipeline. Those failed candidates remain retained.

The staged-evolution APK SHA-256 **8c86dbcd791a824b1402a5561c3ef3926a8f539900d0dcdf95b1563d89726430** completes **128 states × 16 epochs**, **257 × 1,024**, **4,096 × 256** and **65,537 × four**, all with zero bitwise word differences and no nonfinite/invalid records on the real phone. Phone validation layers are **disabled**, so zero validation counters are not a layer pass. Each CPU/GPU comparison starts from a common local snapshot; Windows and Android initialization still uses host-native math, and independently initialized cross-device digests need not match. This is not a claim of cross-device bitwise replay.

Current phone benchmark and capacity

Three fresh-process **4,096-state × 64-epoch, batch 8** phone runs use the same APK and staged schedule. GPU device times are **0.1692938539, 0.1693103924 and 0.177697162 seconds**. The median is **2.64547488125 ms per whole-population epoch**, or **1.548304 million state updates/s**. All reports are healthy and their same-device state/LUT digests agree. Median **app native wall time is 5.818 seconds**: it includes initialization, pipeline setup, upload, advancement, readback and health/digest work, but excludes ADB launch/report polling. It is not the same timer as desktop whole-process wall time. The APK's debug native build, normal USB charging/background conditions, thermal status 0 and battery temperature 37 °C are recorded; this is not a controlled peak-performance or thermal study.

The phone capacity runner completes **14,569,792 full 128-byte records × two epochs**, followed by full readback, digest and health checks. One payload is **1,778.5390625 MiB**; two GPU state copies use **3,557.078125 MiB** of shared RAM. Reported runtime resource allocation is **3,753,000,960 bytes**, including 16 MiB staging, 6 MiB scratch and image/alignment overhead, excluding host snapshots and unreported driver/compiler allocations. Device time is **11.91558916 seconds**, or **2.445501 million state updates/s**. ADB launch-to-report wall time is **77.546 seconds** and maximum submission **41.1456 ms**. The final full-state/LUT digest is 9ae99cb7c910cae5; no nonfinite/out-of-chart records occur.

The limiting policy is **85% of currently available Android physical memory**, accounting for 384 bytes/state at peak (two GPU copies plus one host snapshot), bounded staging/scratch, and a reserve of **226,492,416 bytes**, the larger of the requested 128 MiB and Android's low-memory threshold. Mali exposes no Vulkan memory-budget extension; nominal heap size is not free memory. Available RAM changes during growth, and the runner stops when the refreshed Android policy boundary is reached, without allocation/health failure. This is the largest completed population in that run, not an absolute phone maximum or CPU comparison of all its records. The last capacity report has thermal status 0, battery 36.6 °C and low-memory flag false; after-run Android memory is not a peak-process-memory measurement. Raw evidence is `phone_capacity/summary.json` and `phone_benchmark/summary.json` under the dated results directory.

Current desktop artifact and final GTX measurements

The accepted desktop executable SHA-256 is **f208a48e4cd6c4cad9403736cbbba398c8d283649c1a025418521c3d9de1bb10**. All seven shader payloads are verified inside it. Forced staged execution passes 257 states × 1,024 epochs and 65,537 × four; default monolithic execution passes 257 × 64 and 257 × 1,024. All have zero bitwise differences and core/synchronization validation enabled with zero errors/warnings. Full 4,096-state/64-epoch checkpoints are byte-identical across pre-compaction CPU, compact CPU, current CPU, current monolithic GPU and current staged GPU. The existing 17 Python checks, 30 CPU fixtures, 29 active-body sensitivity checks and CPU checkpoint replay pass. The evidence is consolidated in `2026-09-23/laptop/final_summary.json` and `FINAL_VALIDATION.md`.

For this current executable, the primary three-sample **4,096-state × 64-epoch, batch 8** benchmark uses default monolithic GTX execution. Median GPU device time is **0.157167648 seconds**, or **2.4557445 ms per whole-population epoch** and **1.667926 million state updates/s**; median whole-process time is **0.859 seconds**. CPU medians are **8.1974733 seconds** evolution wall time and **8.218 seconds** whole-process time. Every final state/LUT digest agrees. The GPU median is slower than the preceding pre-compaction artifact's 0.09632352 seconds; these changes demonstrate correctness and phone execution, not a demonstrated speed gain.

The **current GTX capacity** run completes **12,244,544 full records × two epochs** with healthy full readback and digest. A single payload is **1,494.70 MiB**, and its two GPU copies are **2,989.39 MiB**. Device time is **14.0899049 seconds**, giving **1.738059 million state updates/s**. Whole-process time is **36.485 seconds** and maximum submission **38.1315 ms**. Runtime resource allocations total **3,151,412,224 bytes**, including images/padding and staging, excluding host snapshots and unreported driver/compiler allocations. It reaches the 95%-free-budget

minus 128 MiB reserve policy without an allocation/health failure. Validation layers are disabled for timing. This is a completed population under policy, not an absolute maximum or CPU comparison of every capacity record.

A separate three-pair schedule comparison at $4,096 \times 64$ gives **155.282368 ms monolithic** versus **161.960992 ms staged**, so the staged path is **4.3% slower** at the median on this GTX. Every digest agrees; both whole-process medians are 0.750 seconds. This supports retaining the monolithic GTX default. Normal desktop background/clock variation and all samples are preserved in `final_benchmark_summary.json` and `final_schedule_comparison.json`; timer scopes remain distinct.

Claims that can be scanned quickly

Claim	Recorded v0.5.0 status
Mutable LUT is a real integer texture	Implemented and exercised on GTX
All 64 bytes/operator survive texture transfer	Exact roundtrip observed for the recorded multirow case
One-bit controls coexist with full FP32 geometry	Implemented bit flags plus full-width parameters; opcodes remain four-bit
Core LUT data has explicit per-workgroup shared storage	Implemented; 31 records / 1,984 bytes in evolution
LUT stays permanently in hardware cache	Not established
Corrected GTX CPU/GPU agreement	All four broader recorded workloads pass with zero bitwise word differences, including 257 states over 4,096 epochs; general/all-device claim remains unproved
v0.5 phone accuracy	Staged APK passes all listed workloads bitwise, including $257 \times 1,024$, $4,096 \times 256$ and $65,537 \times 4$; initial failures retained
Initial candidate memory capacity	12,490,752 states complete two epochs under the recorded policy; not an absolute hardware maximum
Current-artifact GTX performance/capacity	12,244,544 states $\times$ two epochs at 1.738 million updates/s; 4,096-state $\times$ 64-epoch median 1.668 million updates/s
Current-artifact POCO performance/capacity	14,569,792 states $\times$ two epochs at 2.446 million updates/s; 4,096-state $\times$ 64-epoch median 1.548 million updates/s
Cross-device bitwise initialization/replay from CLI seed	Not guaranteed; initialization still uses host-native math and requires a common imported snapshot for stronger comparison
Fully geometrically self-defined algorithms or a self-growing tree	Not implemented; native integrator, matrix, primitive/wrapping rules, schedule and configured structure remain fixed
Smooth intrinsic Klein dynamics, RK4 order, universal computation or physical quantum behavior	Not established by these implementation checks
Norm conservation	Ideal-transform identity; finite arithmetic and approximation error require measurement

Reproduction and timing scope

Use the exact binary, shader artifacts, numerical profile and arguments identified with each command record. Current examples are instructions, not success claims:

```
dawnwood verify --device "1650 Ti" --count 257 --steps 4096
dawnwood verify --device "1650 Ti" --count 65 --operators 511 --depth 8 --steps 64
dawnwood verify --device "1650 Ti" --count 65537 --steps 4
python tools/measure_capacity.py --binary build/Release/dawnwood.exe --device "1650 Ti" --
```

out results/capacity_run

A capacity result counts only populations that actually advanced all requested epochs and returned healthy full state. Its kernel timer excludes initialization and readback; wall timers have the stated process/app/ADB scopes. Submission duration, actual allocation, state updates per second and available-memory policy answer different questions. Neither a capacity run nor a healthy GPU result certifies CPU/GPU equivalence. Later builds require their own identified evidence; the failed candidate history remains part of this edition.

Appendix B. Sources and edition identity

The user-confirmed source baseline remains the unchanged 18-page *Dawnwood_Interactive_Unified_v0.2.pdf*. Its SHA-256 and the unchanged 24-page source-discussion identity are recorded in `source/formalization_lineage_v0.4.json`. This edition derives its complete fifteen-section structure from `FORMALIZATION_v0.4.md` and adds the changes in `CHANGES_v0.5.md`. `PORTABLE_MATH_v0.5.md` specifies the active elementary-function choices. Shared implementation files and candidate command records are the authority for executable details and observations.

Primary API references: [Khronos GLSL specification](#), [Khronos synchronization examples](#), [Vulkan memory-budget semantics](#), and [Android memory information](#). API contracts explain storage, synchronization and reported limits; they do not substantiate physical or universality claims.